

Memo

记忆化搜索

定义

记忆化搜索是一种通过记录已经遍历过的状态的信息，从而避免对同一状态重复遍历的搜索实现方式。

因为记忆化搜索确保了每个状态只访问一次，它也是一种常见的动态规划实现方式。

引入

▼ [\[NOIP2005\] 采药](#)

山洞里有 n 株不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。给你一段时间，在这段时间里，你可以采到一些草药。让采到的草药的总价值最大。

,

朴素的 [DFS](#) 做法

很容易实现这样一个朴素的搜索做法：在搜索时记录下当前准备选第几个物品、剩余的时间是多少、已经获得的价值是多少这三个参数，然后枚举当前物品是否被选，转移到相应的状态。

▼ 实现



C++Python

```
1 int n, t;
2 int tcost[103], mget[103];
3 int ans = 0;
4
5 void dfs(int pos, int tleft, int tans) {
6     if (tleft < 0) return;
7     if (pos == n + 1) {
8         ans = max(ans, tans);
9         return;
10    }
11    dfs(pos + 1, tleft, tans);
12    dfs(pos + 1, tleft - tcost[pos], tans + mget[pos]);
13 }
14
15 int main() {
16     cin >> t >> n;
17     for (int i = 1; i <= n; i++) cin >> tcost[i] >> mget[i];
18     dfs(1, t, 0);
19     cout << ans << endl;
20     return 0;
21 }
```

```

1 tcost = [0] * 103
2 mget = [0] * 103
3 ans = 0
4
5
6 def dfs(pos, tleft, tans):
7     global ans
8     if tleft < 0:
9         return
10    if pos == n + 1:
11        ans = max(ans, tans)
12        return
13    dfs(pos + 1, tleft, tans)
14    dfs(pos + 1, tleft - tcost[pos], tans + mget[pos])
15
16
17 t, n = map(lambda x: int(x), input().split())
18 for i in range(1, n + 1):
19     tcost[i], mget[i] = map(lambda x: int(x), input().split())
20 dfs(1, t, 0)
21 print(ans)

```

这种做法的时间复杂度是指数级别的，并不能通过本题。

优化

上面的做法为什么效率低下呢？因为同一个状态会被访问多次。

如果我们每查询完一个状态后将该状态的信息存储下来，再次需要访问这个状态就可以直接使用之前计算得到的信息，从而避免重复计算。这充分利用了动态规划中很多问题具有大量重叠子问题的特点，属于用空间换时间的「记忆化」思想。

具体到本题上，我们在朴素的 DFS 的基础上，增加一个数组 `mem` 来记录每个 `dfs(pos, tleft)` 的返回值。刚开始把 `mem` 中每个值都设成 `-1`（代表没求解过）。每次需要访问一个状态时，如果相应状态的值在 `mem` 中为 `-1`，则递归访问该状态。否则我们直接使用 `mem` 中已经存储过的值即可。

通过这样的处理，我们确保了每个状态只会被访问一次，因此该算法的时间复杂度为 $O(n^2)$ 。

▼ 实现



C++Python

```

1 int n, t;
2 int tcost[103], mget[103];
3 int mem[103][1003];
4
5 int dfs(int pos, int tleft) {
6     if (mem[pos][tleft] != -1)
7         return mem[pos][tleft]; // 已经访问过的状态，直接返回之前记录的值
8     if (pos == n + 1) return mem[pos][tleft] = 0;
9     int dfs1, dfs2 = -INF;
10    dfs1 = dfs(pos + 1, tleft);
11    if (tleft >= tcost[pos])
12        dfs2 = dfs(pos + 1, tleft - tcost[pos]) + mget[pos]; // 状态转移
13    return mem[pos][tleft] = max(dfs1, dfs2); // 最后将当前状态的值存下来
14 }
15
16 int main() {
17     memset(mem, -1, sizeof(mem));
18     cin >> t >> n;
19     for (int i = 1; i <= n; i++) cin >> tcost[i] >> mget[i];
20     cout << dfs(1, t) << endl;
21     return 0;
22 }

```

```

1 tcost = [0] * 103
2 mget = [0] * 103
3 mem = [[-1 for i in range(1003)] for j in range(103)]
4
5
6 def dfs(pos, tleft):
7     if mem[pos][tleft] != -1:
8         return mem[pos][tleft]
9     if pos == n + 1:
10        mem[pos][tleft] = 0
11        return mem[pos][tleft]
12    dfs1 = dfs2 = -INF
13    dfs1 = dfs(pos + 1, tleft)
14    if tleft >= tcost[pos]:
15        dfs2 = dfs(pos + 1, tleft - tcost[pos]) + mget[pos]
16    mem[pos][tleft] = max(dfs1, dfs2)
17    return mem[pos][tleft]
18
19
20 t, n = map(lambda x: int(x), input().split())
21 for i in range(1, n + 1):
22     tcost[i], mget[i] = map(lambda x: int(x), input().split())
23 print(dfs(1, t))

```

与递推的联系与区别

在求解动态规划的问题时，记忆化搜索与递推的代码，在形式上是高度类似的。这是由于它们使用了相同的状态表示方式和类似的状态转移。也正因为如此，一般来说两种实现的时间复杂度是一样的。

下面给出的是递推实现的代码（为了方便对比，没有添加滚动数组优化），通过对比可以发现二者在形式上的类似性。

```

1 int n, t, w[105], v[105], f[105][1005];
2
3 int main() {
4     cin >> n >> t;
5     for (int i = 1; i <= n; i++) cin >> w[i] >> v[i];
6     for (int i = 1; i <= n; i++)
7         for (int j = 0; j <= t; j++) {
8             f[i][j] = f[i - 1][j];
9             if (j >= w[i])
10                 f[i][j] = max(f[i][j], f[i - 1][j - w[i]] + v[i]); // 状态转移方程
11         }
12     cout << f[n][t];
13     return 0;
14 }

```

在求解动态规划的问题时，记忆化搜索和递推，都确保了同一状态至多只被求解一次。而它们实现这一点的方式则略有不同：递推通过设置明确的访问顺序来避免重复访问，记忆化搜索虽然没有明确规定访问顺序，但通过给已经访问过的状态打标记的方式，也达到了同样的目的。

与递推相比，记忆化搜索因为不用明确规定访问顺序，在实现难度上有时低于递推，且能比较方便地处理边界情况，这是记忆化搜索的一大优势。但与此同时，记忆化搜索难以使用滚动数组等优化，且由于存在递归，运行效率会低于递推。因此应该视题目选择更适合的实现方式。

如何写记忆化搜索

方法一

1. 把这道题的 dp 状态和方程写出来
2. 根据它们写出 dfs 函数
3. 添加记忆化数组

举例：

（最长上升子序列）

转为



C++Python

```

1 int dfs(int i) {
2     if (mem[i] != -1) return mem[i];
3     int ret = 1;
4     for (int j = 1; j < i; j++)
5         if (a[j] < a[i]) ret = max(ret, dfs(j) + 1);
6     return mem[i] = ret;
7 }
8
9 int main() {
10     memset(mem, -1, sizeof(mem));
11     // 读入部分略去
12     int ret = 0;
13     for (int j = 1; j <= n; j++) {
14         ret = max(ret, dfs(j));
15     }
16     cout << ret << endl;
17 }

```

```
1 def dfs(i):
2     if mem[i] != -1:
3         return mem[i]
4     ret = 1
5     for j in range(1, i):
6         if a[j] < a[i]:
7             ret = max(ret, dfs(j) + 1)
8     mem[i] = ret
9     return mem[i]
```

方法二

1. 写出这道题的暴搜程序（最好是 [dfs](#)）
2. 将这个 dfs 改成「无需外部变量」的 dfs
3. 添加记忆化数组

举例：本文中「采药」的例子

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [interestingLSY](#), [CBW2007](#), [Chrogeek](#), [Enter-tainer](#), [H-J-Granger](#), [Haohu Shen](#), [ImpleLee](#), [lr1d](#), [ksyx](#), [luklapse](#), [Marcythm](#), [Menci](#), [ouuan](#), [shawlleyw](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [Wahacer](#), [Xeonacid](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用